Orario di classe — Prima versione del modello dati

Linguaggio: C#
Livello: classe quarta, Informatica
Obiettivo: rappresentare un orario scolastico usando classi, oggetti e una matrice bidimensionale, senza realizzare ancora la GUI o il motore degli scambi.

1. Il problema

Vogliamo sviluppare un programma che consenta di modificare l'orario di una classe. L'orario è una scacchiera di **5 giorni × 8 ore**, per un totale di **40 celle**. Contiene **34 tessere**, ciascuna relativa a una lezione, e **6 celle vuote**.

Ogni tessera contiene, per ora, soltanto:

- **Materia**, ad esempio `Matematica`;
- **Aula**, ad esempio `A12`.

Le celle vuote possono essere utilizzate per spostare lezioni, ma nell'orario valido **non devono esserci buchi tra due lezioni della stessa giornata**: le ore libere devono trovarsi alla fine del giorno.

L'utente, in una fase successiva, selezionerà due celle e chiederà uno scambio. Un motore separato dalla GUI stabilirà se autorizzarlo.

> **Limite della prima versione:** qui costruiamo soltanto il modello dati. Non controlliamo ancora il numero di tessere, l'assenza di buchi o la validità degli scambi.

2. Organizzazione del progetto

Le tre classi del modello sono raccolte nella cartella `Model`:

```
OrarioClasse/
└── Model/
    ├── Tessera.cs
    ├── Posizione.cs
    └── Orario.cs
```

Queste classi non dipendono da WPF o XAML: potranno essere utilizzate sia da un programma console sia dalla futura interfaccia grafica.

3. Classe Tessera

Una tessera rappresenta una lezione e **non memorizza la propria posizione**. In questo modo la stessa tessera può essere collocata in celle diverse dell'orario.

File `Tessera.cs`:

```
namespace OrarioClasse.Model
{
    public class Tessera
    {
        public string Materia { get; set; }
        public string Aula { get; set; }

        public Tessera(string materia, string aula)
        {
            Materia = materia;
            Aula = aula;
        }
    }
}
```

Le proprietà `Materia` e `Aula` sono pubbliche e utilizzano `get` e `set`: il loro valore può essere letto e modificato. Il **costruttore** riceve i due valori iniziali.

Esempio:

```
Tessera t = new Tessera("Matematica", "A12");
Console.WriteLine(t.Materia); // Matematica
```

4. Classe Posizione

Una posizione identifica una cella attraverso due coordinate: **giorno** e **ora**.

File `Posizione.cs`:

```
namespace OrarioClasse.Model
{
    public class Posizione
    {
        public int Giorno { get; set; }
        public int Ora { get; set; }

        public Posizione(int giorno, int ora)
        {
            Giorno = giorno;
            Ora = ora;
        }
    }
}
```

Gli indici iniziano da zero:

Giorno	Indice
Lunedì	0
Martedì	1
Mercoledì	2
Giovedì	3
Venerdì	4

Le ore hanno indici da **0** (prima ora) a **7** (ottava ora).

```
Posizione p = new Posizione(2, 3);
```

La posizione `p` indica **mercoledì, quarta ora**.

Per ora `Posizione` è una **classe tradizionale**. Due oggetti diversi con le stesse coordinate non risultano automaticamente uguali con l'operatore `==`; per confrontare le coordinate possiamo confrontare separatamente `Giorno` e `Ora`.

5. Classe Orario

L'orario conserva i riferimenti alle tessere in una **matrice bidimensionale**.

File `Orario.cs`:

```
namespace OrarioClasse.Model
{
    public class Orario
    {
        private Tessera[,] celle;

        public Orario()
        {
            celle = new Tessera[5, 8];
        }

        public Tessera Leggi(Posizione posizione)
        {
            return celle[posizione.Giorno, posizione.Ora];
        }

        public void Scrivi(Posizione posizione, Tessera tessera)
        {
            celle[posizione.Giorno, posizione.Ora] = tessera;
        }
    }
}
```

5.1 L'attributo privato `celle`

```
private Tessera[,] celle;
```

`celle` è una matrice bidimensionale di riferimenti a oggetti `Tessera`. È dichiarata `private` per impedire alle altre classi di accedere direttamente alla matrice.

5.2 Il costruttore

```
public Orario()
{
    celle = new Tessera[5, 8];
}
```

`new Tessera[5, 8]` crea **la matrice di 40 posizioni**, non 40 oggetti `Tessera`. Inizialmente tutte le celle contengono `null`, cioè nessun riferimento a una tessera.

5.3 Il metodo `Leggi`

```
public Tessera Leggi(Posizione posizione)
{
    return celle[posizione.Giorno, posizione.Ora];
}
```

Riceve una `Posizione` e restituisce la tessera presente in quella cella. Se la cella è vuota, restituisce `null`.

5.4 Il metodo `Scrivi`

```
public void Scrivi(Posizione posizione, Tessera tessera)
{
    celle[posizione.Giorno, posizione.Ora] = tessera;
}
```

Riceve una posizione e una tessera, quindi assegna alla cella il riferimento alla tessera. È anche possibile passare `null` per svuotare una cella:

```
orario.Scrivi(p, null);
```

Attenzione: questa prima versione permette di sovrascrivere una tessera già presente. La protezione dalle modifiche non autorizzate verrà introdotta con il motore.

6. Il significato di `null` e la configurazione del progetto

In questa versione didattica utilizziamo il tipo tradizionale `Tessera` senza il simbolo `?`. Per evitare gli avvisi del controllo moderno sui riferimenti che possono essere nulli, impostiamo nel file `.csproj`:

```
<Nullable>disable</Nullable>
```

Questa impostazione non impedisce di utilizzare `null`: disabilita l'analisi statica specifica dei *nullable reference types*. Quando studieremo questa funzionalità, potremo riattivarla e utilizzare `Tessera?` per indicare esplicitamente le celle che possono essere vuote.

7. Programma console di prova

Per verificare il modello, creiamo temporaneamente un'applicazione console e inseriamo il seguente codice in `Program.cs`:

```
using System;
using OrarioClasse.Model;

class Program
{
    static void Main()
    {
        Orario orario = new Orario();

        Tessera t1 = new Tessera("Matematica", "A12");
        Tessera t2 = new Tessera("Informatica", "LAB1");

        Posizione p1 = new Posizione(0, 0);
        Posizione p2 = new Posizione(0, 1);

        orario.Scrivi(p1, t1);
        orario.Scrivi(p2, t2);

        Tessera t = orario.Leggi(p1);

        if (t != null)
        {
            Console.WriteLine(t.Materia);
            Console.WriteLine(t.Aula);
        }
    }
}
```

Output atteso:

```
Matematica
A12
```

8. Verifiche da eseguire

Prova	Risultato atteso
Creare un nuovo `Orario`	Tutte le 40 celle contengono `null`
Scrivere una tessera	La cella contiene il riferimento alla tessera
Leggere una cella occupata	Si ottiene la tessera inserita
Leggere una cella vuota	Si ottiene `null`
Scrivere `null` in una cella	La cella diventa vuota
Accedere a indici fuori dai limiti	Si verifica un'eccezione generata automaticamente da C#

Non introduciamo ancora un metodo `ControllaPosizione` né l'istruzione `throw`: se gli indici sono fuori dalla matrice, l'accesso fallisce automaticamente. In seguito introdurremo controlli espliciti.

9. Concetti di programmazione utilizzati

Concetto	Esempio nel progetto
Classe	`Tessera`, `Posizione`, `Orario`
Oggetto	`new Tessera("Matematica", "A12")`
Costruttore	`public Orario()`
Proprietà	`Materia`, `Aula`, `Giorno`, `Ora`
Incapsulamento	`private Tessera[,] celle`
Matrice bidimensionale	`new Tessera[5, 8]`
Riferimento a oggetto	Elemento della matrice che contiene una tessera
`null`	Cella vuota
Metodo con parametro	`Leggi(Posizione posizione)`
Metodo con valore di ritorno	`Tessera Leggi(...)`
Metodo `void`	`Scrivi(...)`

10. Cosa manca e come proseguiremo

Questa versione **non è ancora un gestore di orari completo**. Mancano:

1. Il caricamento di un orario iniziale con esattamente 34 tessere.
2. Il controllo che le sei celle vuote non creino buchi nelle giornate.

3. La richiesta di scambio tra due celle.
4. Un motore che autorizzi o rifiuti lo scambio e lo applichi solo se valido.
5. La GUI WPF con 40 pulsanti disposti in una `Grid`.

La scelta architetturale rimane: **il modello rappresenta i dati; il motore applicherà le regole; la GUI visualizzerà i dati e raccoglierà le richieste dell'utente**.

Fine della prima dispensa — Modello dati.